Week 5 Lab Work

Task 1

```
FIBONACCI
n=10: Recursive time: 0.000219 seconds, Iterative time: 0.000026 seconds
n=20: Recursive time: 0.019374 seconds, Iterative time: 0.000016 seconds
n=30: Recursive time: 1.323719 seconds, Iterative time: 0.000015 seconds

FACTORIAL
n=10: Recursive time: 0.000010 seconds, Iterative time: 0.000007 seconds
n=20: Recursive time: 0.000014 seconds, Iterative time: 0.000010 seconds
n=30: Recursive time: 0.000021 seconds, Iterative time: 0.000015 seconds
```

In the Fibonacci test, the recursive approach's time explodes from near-zero to over 1.3 seconds because it constantly recalculates the same values, whereas the iterative version remains nearly instant by calculating each number only once. In the Factorial test, both methods perform similarly well because the recursive logic is direct and avoids redundant work, with the slight delay in recursion being caused only by the "overhead" of managing multiple function calls. Ultimately, iteration is far superior for sequences like Fibonacci because it follows a linear path rather than a repetitive, branching tree of calculations.

Task 2

Size	Insertion (s)	Merge (s)
100	0.00342	0.00271
1000	0.16646	0.01695
5000	4.17477	0.10228

These results highlight the massive difference between quadratic and log-linear time complexity in sorting algorithms. As your data size grows from 100 to 5,000, Insertion Sort slows down dramatically (from 0.003s to over 4.17s) because its nature means that doubling the input roughly quadruples the work. In contrast, Merge Sort remains highly efficient (finishing in just 0.10s for 5,000 items) because its "divide and conquer" strategy scales much more gracefully. While Insertion Sort is simple to write and performs fine for tiny lists, Merge Sort is the superior choice for any real-world dataset due to its predictable and vastly faster scaling.

Task 3

1. Test of Tower of Hanoi

```
Enter number of disks: 3
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
```

The test went as expected, with the number of moves increasing exponentially as the number of disks increase to move. It as well appears to increase by almost double as it the number disks increase as well.

```
Enter number of disks: 5
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 4 from A to C
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 3 from B to C
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 5 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 3 from C to A
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 4 from C to B
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
```

2. Number of Moves

Test 1:

```
Enter number of disks: 5
The moves are:
Number of moves is 31
```

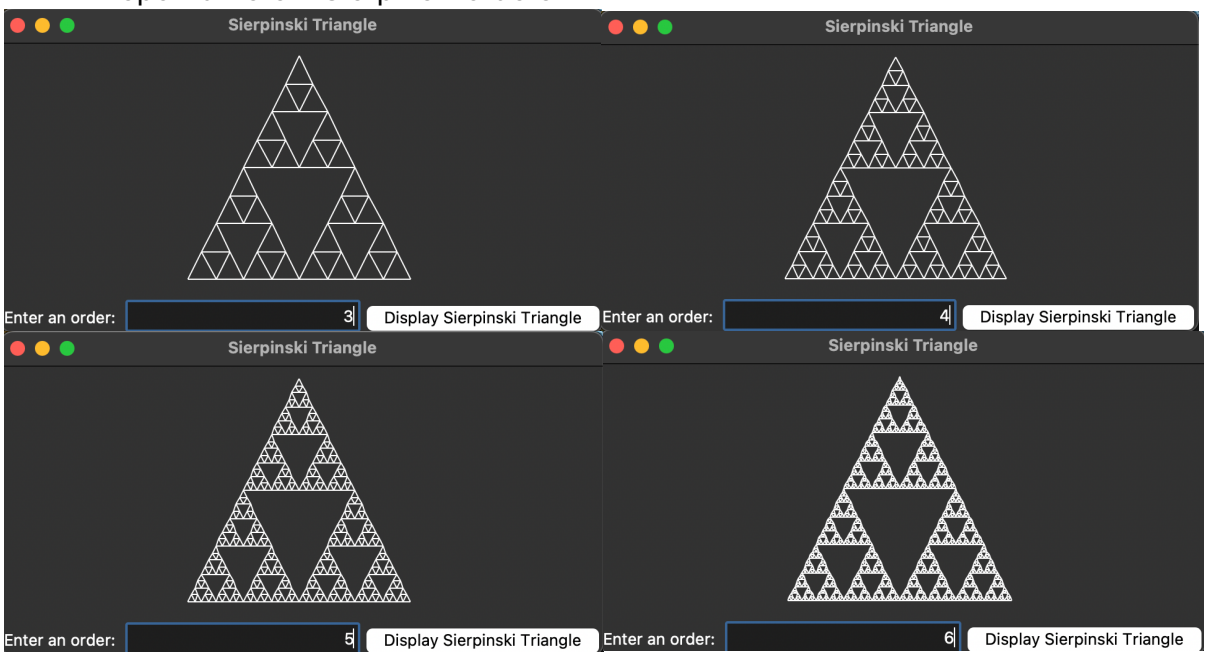
Test 2:

```
Enter number of disks: 7
The moves are:
Number of moves is 127
```

The results of the Tower of Hanoi illustrated the exponential growth characteristic of this recursive algorithm. With 5 disks requiring 31 moves and 7 disks requiring 127 moves, the data perfectly follows the mathematical formula $2^n - 1$ $(2^5 - 1 = 31\ and\ 2^7 - 1 = 127)$.

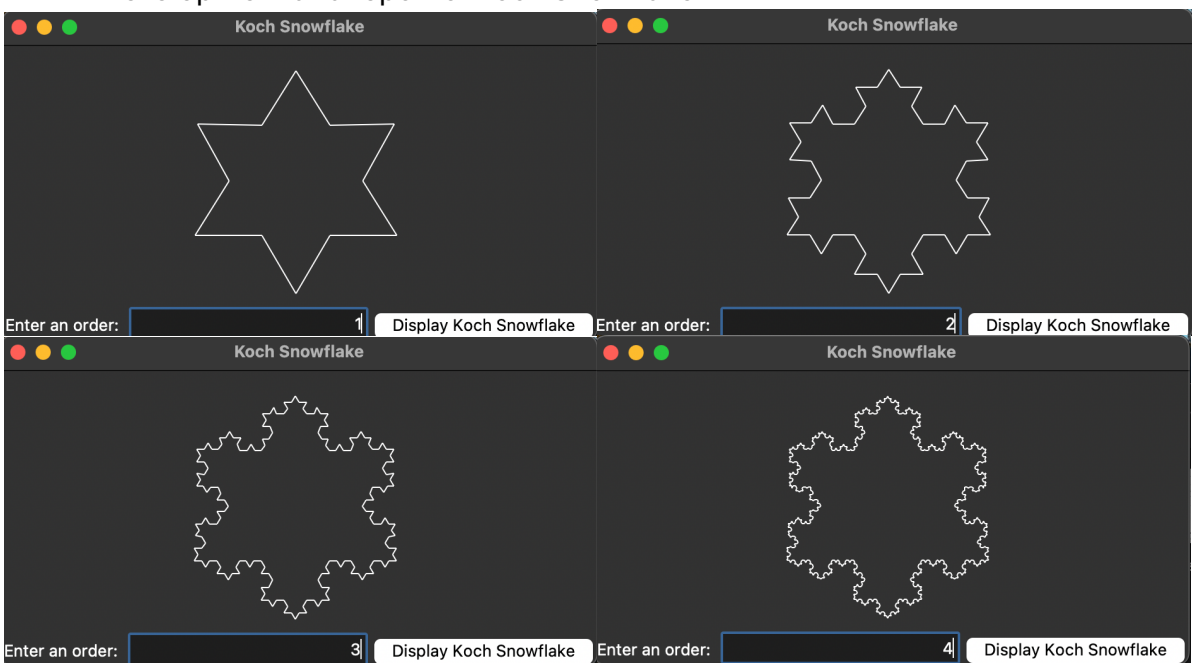
Task 4

1. Report different Sierpinski orders



As you can see with the order of the Sierpinski triangles (3,4,5,6) it continuously became further dense as the recursive algorithm added further triangles. To the point where the 6th order became so dense that the gaps between lines started to disappear. Even if I started with the order 5 and went up it would be too difficult to have any proper analysis due to the density.

2. Development and report of Koch Snowflake



The result of the Koch Snowflake demonstrated the recursive nature of fractals, where a simple geometric rule is applied repeatedly to create increasing complexity. Starting from a basic equilateral triangle (Order 0 which is not displayed), each subsequent

order replaces every straight-line segment with four smaller segments, forming a new peak in the centre.